

NEXA BANK

Transactions, Transaction Logs & Beneficiaries

RAG & AI Agent Knowledge Base

This knowledge base defines the operational concepts, workflows, business rules, audit expectations, beneficiary management, and support guidance for Nexa Bank's transaction domain. It is intended to provide grounded context for a RAG-powered banking assistant and AI agent used in the Nexa Bank demo team project.

Scope: transactions, transfers, transaction status and lifecycle, validation, failures and recovery, transaction logs and audit trails, beneficiary management, security-aware chatbot responses, and agent decision boundaries.

Nexa Bank Demo Team Project

1. Purpose and Role in the RAG System

The Nexa Bank Transaction Knowledge Base is the authoritative business-context layer for transaction-related questions handled by the RAG system. Retrieval should ground the AI agent in these definitions and rules before it produces an answer or decides whether a tool/action is appropriate.

Knowledge-base principle: retrieved knowledge explains what Nexa Bank means by a transaction, status, beneficiary, validation rule, audit event, and recovery path. Live customer-specific facts such as an actual balance, current transaction status, beneficiary record, or transaction history must come from authorized application tools or APIs rather than being invented.

Recommended RAG behaviour

- Retrieve the most relevant transaction-domain passages before answering domain questions.
- Use exact transaction IDs, beneficiary IDs, and other identifiers only when they are present in retrieved or tool-provided data.
- Separate policy/business knowledge from live account data.
- Cite or expose the supporting knowledge source when the application design requires source attribution.
- If required information is unavailable, state that it is unavailable rather than guessing.

2. Transactions

A transaction is a recorded movement of funds or financial event associated with a Nexa Bank account. Supported transaction scenarios include internal transfers, UK local transfers, deposits, withdrawals, card purchases, fees, refunds, reversals, and scheduled payments, subject to the capabilities implemented by the application.

Core transaction information

- Transaction ID — unique identifier for the transaction.
- Source account ID — account from which funds originate, where applicable.
- Destination account ID — receiving account, where applicable.
- Beneficiary ID — saved beneficiary used by a transfer, where applicable.
- Amount and currency — monetary value and currency of the transaction.
- Transaction type — describes the nature of the financial event.
- Description and reference — human-readable and reconciliation information.
- Initiating channel — web, mobile, API, branch, scheduled process, or another supported channel.
- Timestamps — creation, submission, processing, and completion times.
- Current status — the latest lifecycle state.
- Failure/rejection reason — structured reason when processing does not succeed.
- Correlation ID — identifier used to trace related requests and system events.
- Linked transaction IDs — references to refunds, adjustments, replacements, or reversals.

Transaction integrity

Original transaction records should remain traceable after creation. Corrections should normally be represented by linked adjustments, refunds, replacement transactions, or reversals instead of silently overwriting the original event. This preserves financial history and makes the transaction explainable to customers, support staff, administrators, and the audit process.

3. Transaction Types

Type	Meaning	Typical direction
DEBIT	Funds leave the source account.	Outgoing
CREDIT	Funds enter the receiving account.	Incoming
INTERNAL TRANSFER	Transfer between eligible Nexa Bank accounts.	Debit + credit
UK LOCAL TRANSFER	Transfer to an eligible UK destination.	Outgoing
DEPOSIT	Funds are added to an account.	Incoming
WITHDRAWAL	Funds are removed from an account.	Outgoing
REFUND	Funds returned in relation to an earlier transaction.	Incoming
REVERSAL	Linked event that offsets a previous completed transaction.	Context-dependent
FEE	Charge associated with a supported service or transaction.	Outgoing

Transaction types should be interpreted according to the actual capabilities and data model exposed by the Nexa Bank application.

4. Transaction Lifecycle and Statuses

A standard transaction lifecycle is:

INITIATED → VALIDATING → PROCESSING → COMPLETED

- **INITIATED:** the request has been received but processing has not started.
- **VALIDATING:** account, beneficiary, amount, authorization, limits, duplication, and other applicable checks are being performed.
- **PENDING:** the transaction is waiting for a scheduled time, review, or supported downstream response.
- **PROCESSING:** initial checks have passed and the funds movement is being processed.
- **COMPLETED:** processing finished successfully according to the transaction workflow.
- **FAILED:** processing could not finish successfully.
- **REJECTED:** a validation or business rule deliberately declined the request.
- **CANCELLED:** an authorized action stopped the transaction before completion.
- **REVERSED:** a later linked event offsets a previously completed transaction.

Important: a completed transaction should not silently become FAILED. If a completed transaction requires correction, the history should show the linked reversal, adjustment, refund, or replacement event.

5. Transaction Validation

Before processing a transaction, Nexa Bank may validate:

- Required fields are present and correctly formatted.
- Amount is greater than zero and uses supported precision.
- Currency is supported for the selected transaction type.
- Source account exists, belongs to the requester, and is active.
- Available balance is sufficient for the transaction and any applicable fee.
- Beneficiary exists, is enabled, and matches the submitted transfer details.
- Applicable transaction, daily, or monthly limits are not exceeded.
- The requester is authenticated and authorized.

- The request is not expired and has not already been processed as a duplicate.
- Scheduled dates are valid where scheduling is supported.

Validation outcomes

- **PASS:** required checks succeeded.
- **WARNING:** processing may continue but an informational condition exists.
- **REJECT:** a business or validation rule prevents processing.
- **ERROR:** an unexpected technical or system condition prevents normal processing.

Customer-facing explanations should be useful without exposing sensitive account information. For example: *"The transfer could not be completed because the requested amount exceeds the applicable daily transfer limit."*

6. Failure, Rejection and Recovery Scenarios

Scenario	Meaning	Agent guidance
Insufficient funds	Available balance is below the required amount.	Explain that funds are insufficient; do not expose unrelated balance data.
Unknown beneficiary	Beneficiary ID cannot be found.	Ask the user to select or create a valid beneficiary.
Disabled beneficiary	Beneficiary exists but cannot receive new transfers.	Do not initiate the transfer; explain that the beneficiary is unavailable.
Invalid account state	Account is closed, suspended, or otherwise ineligible.	Do not attempt the transaction; direct the user to supported account servicing.
Limit exceeded	Applicable transaction or period limit was exceeded.	Explain the relevant limit condition if available.
Duplicate request	A matching request was recently submitted.	Avoid creating a second transaction; retrieve the existing transaction if authorized.
Invalid amount/currency	Amount, precision, or currency is unsupported.	Ask for a corrected amount or supported currency.
Expired authorization	Authorization is no longer valid.	Require the supported authorization flow again.
Processing timeout	Downstream processing did not respond within the expected period.	Check transaction state before suggesting retry.
System interruption	Internal component was unavailable.	Do not claim funds moved unless confirmed by transaction data.
Concurrency conflict	Relevant account/beneficiary data changed during processing.	Refresh authoritative state before retrying.

For any failure, the AI agent should distinguish between **transaction status** and **money movement**. A technical error does not by itself prove that funds did or did not move. The agent should rely on authoritative transaction/account data when answering that question.

7. Transaction Logs and Audit Trails

A transaction log records operational events associated with a transaction. An audit trail records who or what performed an action, when it happened, and what changed. Nexa Bank audit records should be chronological, append-only where applicable, traceable, and restricted to information the requesting user is authorized to see.

Audit event fields

- Audit event ID
- Transaction ID
- Event type
- Previous status and new status
- Actor type — customer, administrator, service, scheduler, or another supported actor
- Actor reference
- Timestamp
- Reason or explanation
- Request correlation ID
- Request channel and result code

- Selected before-and-after values, with sensitive values masked
- Linked event ID for refunds, adjustments, or reversals

Example audit sequence

- A customer creates a transfer through the web or mobile channel.
- The request enters validation.
- The beneficiary is confirmed as eligible.
- Account, amount, authorization, and limit checks pass.
- The transaction moves to PROCESSING.
- Funds movement completes.
- A COMPLETED event is recorded.
- The final audit record retains the correlation and transaction references needed for tracing.

8. Beneficiary Management

A beneficiary is a saved recipient used for eligible future payments or transfers. A customer or account may have multiple beneficiaries. A beneficiary can be referenced by many transactions. Historical transactions must remain explainable even when a beneficiary is later disabled, removed, or edited.

Beneficiary record

- Beneficiary ID
- Display name
- Masked destination reference
- Account or payment identifier where applicable
- Preferred currency
- Owning customer/account relationship
- Verification state
- Current status
- Creation timestamp
- Last updated timestamp

Common beneficiary actions

- Add a beneficiary
- Complete or review beneficiary verification
- Edit permitted beneficiary details
- Enable or disable a beneficiary
- Remove a beneficiary from future use
- Search or filter beneficiaries by name, status, or other supported fields

Changes to beneficiary records should create appropriate audit events.

9. Beneficiary Statuses and Rules

Status	Meaning	Can receive new transfers?
ACTIVE	Beneficiary is available for eligible transactions.	Yes
PENDING_VERIFICATION	Beneficiary has been created but verification is incomplete.	No
DISABLED	Beneficiary is temporarily unavailable.	No
REMOVED	Beneficiary is unavailable for future use; historical references remain	No

A transfer should not use a beneficiary in PENDING_VERIFICATION, DISABLED, or REMOVED status. Removing a beneficiary must not erase completed transactions that used that beneficiary. Historical transaction data should preserve the recipient reference or transaction-time details required for auditability.

10. Relationships Between Transactions, Logs and Beneficiaries

- Customers own or are associated with accounts.
- Accounts create, send, receive, or are otherwise associated with transactions.
- Beneficiaries belong to a customer/account relationship and may be referenced by transfers.
- A transaction can have many transaction-log/audit events.
- Each transaction-log event belongs to a transaction.
- Refunds, reversals, adjustments, and replacements may link to an original transaction.
- Authentication and authorization determine which records and actions a user may access.
- Administrators/support roles may have broader visibility according to the application's authorization model.

11. RAG Retrieval Guidance

High-value retrieval topics

- Transaction definitions and types
- Transaction lifecycle and status meanings
- Validation and rejection rules
- Failure and recovery scenarios
- Transaction logs and audit events
- Beneficiary statuses and lifecycle
- Relationships between transaction, account, beneficiary, and audit records
- Frequently asked customer questions

Chunking recommendations

- Keep each status definition with its meaning and allowed transition context.
- Keep failure scenarios together with their customer-facing recovery guidance.
- Keep beneficiary statuses together with the rule that inactive beneficiaries cannot receive new transfers.
- Keep audit-field definitions close to examples of audit events.
- Use descriptive section headings so retrieval can identify the domain and topic.
- Avoid putting unrelated policies in the same chunk.

Metadata recommendations

- domain = transactions

- entity = transaction | transaction_log | beneficiary
- topic = lifecycle | validation | failure | audit | status | management | faq
- source = Nexa Bank Transaction Knowledge Base
- version = document version maintained by the project team

12. AI Agent Decision Boundaries

The RAG component provides knowledge. The agent should use authorized tools for customer-specific operations. The distinction is critical:

Question/action	Primary source
What does PENDING mean?	RAG knowledge base
Why can a beneficiary be disabled?	RAG knowledge base + live beneficiary state when relevant
What is transaction TX-1001's current status?	Authorized transaction API/tool
Did TX-1001 change the account balance?	Authorized transaction/account API/tool
Who changed TX-1001's status?	Authorized transaction-log/audit API/tool
Create a transfer	Authorized transaction tool + RAG for rules/explanations
Add/disable beneficiary	Authorized beneficiary tool + RAG for workflow rules
Invent a missing transaction, balance, limit, or person	Never

13. Security and Privacy Rules for the Agent

- Never reveal full account numbers, payment identifiers, authentication secrets, or other sensitive credentials unless explicitly permitted by the application's security design.
- Mask sensitive identifiers in customer-facing responses.
- Do not expose internal service names, database details, access tokens, or security controls unnecessarily.
- Confirm authorization before retrieving or changing another customer's transaction or beneficiary information.
- Do not infer that a user owns an account merely because they supplied an account number.
- Use authoritative system data for customer-specific facts.
- When a request is unauthorized, explain the access limitation without revealing protected data.

14. Frequently Asked Questions for the RAG Agent

What does PENDING mean?

PENDING means the transaction is waiting for a scheduled time, review, or supported downstream response. The live transaction record should be checked for the exact reason.

What is the difference between FAILED and REJECTED?

REJECTED means a validation or business rule deliberately declined the request. FAILED means processing could not finish successfully.

What is CANCELLED?

CANCELLED means an authorized action stopped the transaction before completion.

What is REVERSED?

REVERSED means a later linked event offsets a transaction that had previously completed.

Can I transfer to a beneficiary awaiting verification?

No. A PENDING_VERIFICATION beneficiary is not eligible for a new transfer.

What happens when a beneficiary is removed?

The beneficiary becomes unavailable for future use, while historical transactions that referenced the beneficiary remain traceable.

Who changed my transaction status?

The agent should retrieve the transaction's authorized audit/log records and identify the actor recorded for the relevant status change.

Did my money move after a failure?

The agent should not infer this from the word FAILED alone. It should check authoritative transaction and account state.

15. Example User Questions

- What does PENDING mean for my transfer?
- Why did my transfer fail, and did the funds move?
- What is the difference between FAILED, REJECTED, CANCELLED, and REVERSED?
- Show me the status history for transaction TX-1001.
- Who changed TX-1001 from PENDING to COMPLETED?
- When did my transaction enter PROCESSING?
- What information is stored in a transaction log?
- Why can't I transfer to this beneficiary?
- How do I add a beneficiary?
- Can I edit or disable a beneficiary?
- Can I transfer to a beneficiary awaiting verification?
- What happens to my previous transactions after a beneficiary is removed?
- Why was my transaction rejected?
- Was a fee applied to my transfer?
- Can I retry this transaction?

16. Grounding and Fallback Policy

When information is not present in this knowledge base or in authorized live application data, the Nexa Bank AI assistant must say that the information is unavailable. It must not invent transaction records, account balances, beneficiary details, limits, fees, dates, people, statuses, or policy decisions.

For transaction-specific questions, the preferred response flow is: **retrieve relevant business knowledge → identify required live data → check authorization → call the appropriate tool/API → interpret the returned state → answer clearly → state uncertainty or limitations where applicable.**

This separation allows the RAG layer to explain Nexa Bank's transaction domain while keeping real-time customer data and state changes under the control of authorized backend services.

17. Implementation Notes for the Demo Team

- Treat this document as business/domain knowledge, not as a source of live customer records.
- Keep transaction APIs responsible for authoritative transaction state.
- Keep beneficiary APIs responsible for authoritative beneficiary state.
- Keep transaction-log/audit APIs responsible for authoritative event history.
- Use stable IDs such as `transaction_id`, `beneficiary_id`, and `audit_event_id` for retrieval and tracing.
- Use correlation IDs to connect user requests, backend operations, asynchronous jobs, and related events.
- Design agent tools with explicit authorization checks and narrow permissions.
- Return structured tool results so the agent can distinguish status, reason, money movement, and audit history.
- Log agent actions separately from customer transaction events when appropriate, so AI activity remains traceable.
- Version the knowledge base so changes to transaction rules can be identified and retrieved consistently.

18. Knowledge Base Summary

Nexa Bank's transaction domain can be understood through three connected areas: **transactions** describe financial events and their lifecycle; **transaction logs and audit trails** explain what happened, when, and by whom; and **beneficiaries** define saved recipients and their eligibility for transfers. A RAG-powered AI agent should use this knowledge to explain concepts and guide workflows, while authorized tools provide the live customer-specific truth.

Core rule for the AI agent: knowledge explains the banking process; tools verify and change live banking state; neither component should fabricate missing information.